

Solving a Risky-Debt Corporate Finance Model with Deep Learning

a toy example on the possibility of applying machine learning to finance problems

Chak Wong

June 23, 2026

Abstract

These notes explain how to adapt the deep-learning method of [Maliar et al. \(2021\)](#) to the risky-debt dynamic corporate finance model in Section 3.6 of [Strebulaev and Whited \(2012\)](#). The goal is not to prove that a particular neural network will always converge. The goal is more modest and more useful for computation: we rewrite the firm's recursive problem, the limited-liability default condition, and the lender's zero-profit debt-pricing condition as residual equations that can be minimized by stochastic gradient descent. The exposition is written for early undergraduates. It uses functions, sums, informal derivatives, expected values, and a small amount of optimization language; each specialized idea is introduced before it is used.

Contents

1	What problem are we solving?	2
2	Expected values and the shock process	3
3	Warm-up: the basic investment Bellman equation	3
4	The risky-debt model	4
4.1	Cash flow with risky debt	4
4.2	Default and recovery	5
5	Debt pricing	6
6	Writing default as equations	8
7	Neural-network approximations	8
8	Residuals for the risky-debt model	9
8.1	Bellman and default residual	9
8.2	Debt-pricing residual	9
8.3	Policy maximization residual	10
8.4	Euler residuals from first-order conditions	11
8.5	Training with Euler residuals and nonsmooth regions	12
8.6	Hard constraints as output transformations	14
9	The training objective	16
9.1	Replacing fixed weights by multi-objective training	17

10 Comparison with Duarte and Fonseca’s structural-estimation algorithm	20
10.1 What their algorithm does	20
10.2 How this compares with the first approach in this note	21
10.3 Is it suitable for the risky-debt problem?	22
11 Parameter-conditioned Maliar–Maliar–Winant solvers	23
11.1 Why parameter sampling is difficult	24
12 An alternative estimation route: state-space models, filtering, and HMC	26
12.1 From solved policies to a state-space model	26
12.2 Filtering and HMC	27
12.3 Can we do this for the risky-debt model?	27
13 A full-scale state-space specification	28
13.1 Data and parameters	28
13.2 Latent state and initial distribution	29
13.3 Solved policy transition	29
13.4 Measurement equations	30
13.5 Exact filtering specification	30
13.6 Smoothed HMC specification	31
13.7 How this specification addresses the five problems	31
14 Likelihood filtering versus moment matching	32
14.1 Advantages of moment matching	33
14.2 Advantages of likelihood filtering and HMC	33
14.3 Side-by-side comparison	34
14.4 Which one should we use?	34
15 The all-in-one expectation idea	35
16 A concrete algorithm	35
17 What this derivation does and does not prove	36

1 What problem are we solving?

Dynamic corporate finance models describe a firm that makes decisions today, then faces uncertainty tomorrow. In the simple investment model of [Strebulaev and Whited \(2012, Section 3.1\)](#), the firm chooses next period’s capital stock. In the risky-debt model of [Strebulaev and Whited \(2012, Section 3.6\)](#), the firm chooses both next period’s capital and next period’s debt, and debt is risky because the firm may default.

The difficult object is the *value function*. A value function is a rule that assigns a number to each state of the firm. For example,

$$V(k, b, z)$$

means the equity value of a firm with capital k , debt b , and productivity shock z . The state variables are:

- k : beginning-of-period capital;

- b : debt due at the beginning of the period, with negative b interpreted as cash;
- z : a productivity or profitability shock.

Maliar et al. (2021, Definition 2.8 and Algorithm 1) show how a dynamic economic model can be turned into a residual-minimization problem. The broad idea is:

1. Write the economic model as equations.
2. Approximate unknown functions, such as V , by neural networks.
3. Measure how badly the neural networks violate the model equations.
4. Train the networks to make those violations small.

The central question of this note is: can we do this for the risky-debt model? The answer is yes, with one important qualification. We must not minimize only the Bellman equation. We must also impose the lender's debt-pricing equation, because the risky interest rate is itself an equilibrium object.

2 Expected values and the shock process

The firm does not know next period's shock z' when it makes today's choices. Following Strebulaev and Whited (2012, eq. (3.9)), assume that the shock follows the log autoregression

$$\log z' = \rho \log z + \varepsilon', \quad (1)$$

where ρ is a persistence parameter and ε' is a random innovation. If we know today's z , then eq. (1) gives the distribution of tomorrow's z' .

If $F(z')$ is any function of tomorrow's shock, then

$$\mathbb{E}[F(z') \mid z]$$

means the average value of $F(z')$, using the probabilities implied by today's state z . If the shock has finitely many possible next values

$$z'_1, \dots, z'_n$$

with probabilities $p_1, \dots, p_n$, then

$$\mathbb{E}[F(z') \mid z] = p_1 F(z'_1) + \dots + p_n F(z'_n).$$

In the continuous case, the sum becomes an integral. These notes use expectation notation because it covers both cases.

3 Warm-up: the basic investment Bellman equation

In the basic model of Strebulaev and Whited (2012, Section 3.1), the firm has capital k , chooses next period's capital k' , and therefore invests

$$I = k' - (1 - \delta)k, \quad (2)$$

where δ is the depreciation rate. The one-period cash flow is

$$e(k, k', z) = \pi(k, z) - \psi(k' - (1 - \delta)k, k) - (k' - (1 - \delta)k). \quad (3)$$

Here $\pi(k, z)$ is profit and $\psi(I, k)$ is an adjustment cost.

Let

$$\beta = \frac{1}{1 + r},$$

where r is the risk-free interest rate. The number β is the one-period discount factor.

Proposition 1 (Bellman equation for the basic investment model). *Suppose the firm maximizes the expected discounted sum of future cash flows. Then the value function satisfies*

$$V(k, z) = \max_{k'} \{e(k, k', z) + \beta \mathbb{E}[V(k', z') \mid z]\}. \quad (4)$$

Proof. The value of the firm is the value of today's cash flow plus the discounted value of all future cash flows. If the firm chooses k' , today's cash flow is $e(k, k', z)$. Tomorrow the firm will be in state (k', z') , so tomorrow's value is $V(k', z')$. Because z' is random, the firm uses the average value $\mathbb{E}[V(k', z') \mid z]$. Because tomorrow's value is one period in the future, it is multiplied by β . Finally, the firm chooses the k' that gives the largest total. This gives eq. (4). $\square$

This has the same recursive structure as [Strebulaev and Whited \(2012, eq. \(3.10\)\)](#), but written with β instead of $1/(1 + r)$.

4 The risky-debt model

Now add one-period debt. The firm starts the period with debt b , chooses next period's capital k' , and chooses next period's debt b' . The important new feature in [Strebulaev and Whited \(2012, Section 3.6\)](#) is that debt is risky. A lender will charge a risky interest rate

$$\tilde{r}(z, k', b')$$

that depends on today's shock z and on the firm's promised next-period capital and debt (k', b') .

4.1 Cash flow with risky debt

[Strebulaev and Whited \(2012, eq. \(3.26\)\)](#) define the one-period cash flow as

$$\begin{aligned} e(k, k', b, b', z; \tilde{r}) = & (1 - \tau)\pi(k, z) - \psi(k' - (1 - \delta)k, k) - (k' - (1 - \delta)k) \\ & + \frac{b'}{1 + \tilde{r}(z, k', b')} + \frac{\tau \tilde{r}(z, k', b') b'}{(1 + \tilde{r}(z, k', b'))(1 + r)} - b. \end{aligned} \quad (5)$$

The terms have simple interpretations:

- $(1 - \tau)\pi(k, z)$ is after-tax profit.
- $\psi(k' - (1 - \delta)k, k)$ is the adjustment cost of investment.
- $k' - (1 - \delta)k$ is physical investment.
- $b'/(1 + \tilde{r})$ is the amount of cash raised today by issuing a promise to repay b' next period.

- The term with $\tau\tilde{r}b'$ is the present value of the interest tax deduction used in the Section 3.6 model.
- The final $-b$ is repayment of debt already due today.

The firm may also face costly external equity finance. To avoid confusing the cash-flow function $e(\cdot)$ with the argument of the financing-cost function, write a generic pre-financing cash flow as x . Let $\eta(x)$ be the additional value effect of external financing costs. In many corporate finance models, $\eta(x) = 0$ when $x \geq 0$, because the firm is distributing cash, and $\eta(x) \leq 0$ when $x < 0$, because the firm must raise costly outside equity. The exact functional form is not needed for the derivation below.

4.2 Default and recovery

If the firm defaults next period, the lender recovers

$$R(k', z') = (1 - \alpha)((1 - \tau)\pi(k', z') + (1 - \delta)k'), \quad (6)$$

where $\alpha \in [0, 1]$ measures deadweight default costs. This is the recovery formula stated in [Strebulaev and Whited \(2012, Section 3.6\)](#).

Limited liability means equityholders can walk away instead of accepting a negative equity value. Thus equity value is never below zero.

Remark 1 (Do not mix up equity value and debt value). *This is the point at which the notation can become confusing. In these notes, $V(k, b, z)$ is the equity value. It is the value to shareholders, not the value of the lender's debt claim and not the total value of all claims on the firm. Therefore, if the firm defaults, shareholders receive zero by limited liability. The recovery $R(k', z')$ in eq. (6) is paid to lenders, so it belongs in the debt-pricing equation, not as a positive payoff inside the equity continuation value.*

If instead we were writing a value function for lenders, or a total firm value function that adds equity and debt together, then default recovery would appear directly in that object's payoff. The Section 3.6 Bellman equation is an equity Bellman equation, so default gives equityholders zero while recovery affects equity indirectly through the risky interest rate.

Definition 1 (Policy function). *A policy function is a rule that tells the firm what to do at every possible state. In the risky-debt model, a stationary Markov policy is a pair of functions*

$$h(k, b, z) = (h^k(k, b, z), h^b(k, b, z)),$$

where $h^k(k, b, z)$ is the next-period capital chosen at state (k, b, z) , and $h^b(k, b, z)$ is the next-period debt chosen at that state. The word stationary means that the same rule is used in every period. The word Markov means that the rule depends only on the current state (k, b, z) , not on the full past history.

Definition 2 (Policy-specific and optimal going-concern values). *There are two different objects that should not be confused.*

First, suppose the firm is committed to a particular policy h . Given a policy-specific equity value V^h and a risky rate $\tilde{r}$, define the value of taking action (k', b') today and then following policy h from tomorrow onward by

$$Q^h(k, b, z; k', b') = e(k, k', b, b', z; \tilde{r}) + \eta(e(k, k', b, b', z; \tilde{r})) + \beta \mathbb{E}[V^h(k', b', z') \mid z]. \quad (7)$$

If the firm follows the policy h today, it chooses $(h^k(k, b, z), h^b(k, b, z))$, so its policy-specific going-concern value is

$$G^h(k, b, z) = Q^h(k, b, z; h^k(k, b, z), h^b(k, b, z)). \quad (8)$$

Policy evaluation therefore uses no maximization over actions:

$$V^h(k, b, z) = \max\{0, G^h(k, b, z)\}. \quad (9)$$

Second, if we are solving for the best policy, we use Bellman optimality. Let V^* denote the optimal equity value. Define

$$Q^*(k, b, z; k', b') = e(k, k', b, b', z; \tilde{r}) + \eta(e(k, k', b, b', z; \tilde{r})) + \beta \mathbb{E}[V^*(k', b', z') \mid z]. \quad (10)$$

This is the optimal action-value expression. It asks: if the firm takes the particular action (k', b') today, and then behaves optimally from tomorrow onward, what is today's equity value? The continuation term $\mathbb{E}[V^*(k', b', z') \mid z]$ is already an equity value with limited liability built into it. It averages next period's shareholder value, which is zero in future default states. Define the best going-concern equity value by

$$G^*(k, b, z) = \max_{k', b'} Q^*(k, b, z; k', b'). \quad (11)$$

When the maximum is attained, an optimal policy h^* chooses an action that attains it at every state:

$$h^*(k, b, z) \in \arg \max_{k', b'} Q^*(k, b, z; k', b').$$

Remark 2 (Connection with dynamic programming and reinforcement learning). In reinforcement-learning notation, Q^h is the action-value function for a fixed policy, while Q^* is the optimal action-value function. Evaluating an existing policy uses Q^h and V^h . Improving or solving for the optimal policy uses the maximization in G^* . To match the notation in [Strebulaev and Whited \(2012, Section 3.6\)](#), the rest of these notes often drops the star and writes V, Q, G, h for the optimal objects V^*, Q^*, G^*, h^* . Whenever we evaluate a fixed nonoptimal policy, we keep the superscript h .

Proposition 2 (Risky-debt Bellman equation). In the risky-debt model, optimal equity value satisfies

$$V^*(k, b, z) = \max\{0, G^*(k, b, z)\}. \quad (12)$$

Proof. If the firm continues, the best value it can obtain is $G^*(k, b, z)$, because G^* is defined as the best equity value over all choices of (k', b') , followed by optimal behavior in the future. If that value is negative, equityholders prefer to default and receive zero rather than keep a negative-value equity claim. Therefore equityholders choose the larger of zero and the optimal going-concern equity value. This gives eq. (12), matching [Strebulaev and Whited \(2012, eq. \(3.28\)\)](#). The lender's recovery is not missing; it enters the lender's pricing equation below and changes current equity value through the risky rate $\tilde{r}(z, k', b')$. It also matters recursively because future equity values are computed using future debt-pricing equations. $\square$

5 Debt pricing

The risky interest rate is not chosen freely by the firm. Competitive lenders must break even. If the lender gives the firm cash today in exchange for a promise b' , then the expected payoff next period must equal what the lender could earn at the risk-free rate. In this note, as in the Section

3.6 formula, b' denotes the debt position chosen today for next period. We use the paper's payoff convention directly: when the firm is solvent next period, lenders receive $b'(1 + \tilde{r})$; when the firm defaults, lenders receive the recovery value.

Let

$$D(k', b', z') = \begin{cases} 1, & \text{if the firm defaults next period,} \\ 0, & \text{if the firm remains solvent next period.} \end{cases}$$

Equivalently, using eq. (12), the next-period state is a default state when its best going-concern equity value is nonpositive:

$$D(k', b', z') = \mathbf{1}\{G^*(k', b', z') \leq 0\}. \quad (13)$$

Proposition 3 (Zero-profit risky debt pricing). *For $b' > 0$, the zero-profit condition for the risky debt rate is*

$$b'(1 + r) = \mathbb{E} [D(k', b', z')R(k', z') + (1 - D(k', b', z'))b'(1 + \tilde{r}(z, k', b')) \mid z]. \quad (14)$$

Proof. The left side, $b'(1 + r)$, is the payoff a lender requires next period to earn the risk-free return on the chosen debt position b' . On the right side, there are two cases. If the firm defaults, $D = 1$, and the lender receives the recovery value $R(k', z')$. If the firm is solvent, then $D = 0$, and the lender receives the promised payoff $b'(1 + \tilde{r})$. The lender prices debt so that the expected payoff equals the required risk-free payoff. This is [Strebulaev and Whited \(2012, eq. \(3.27\)\)](#) written with an indicator variable. $\square$

Remark 3 (Solving explicitly for the risky rate). *If $b' > 0$ and the probability of solvency is positive, then eq. (14) can be rearranged as*

$$1 + \tilde{r}(z, k', b') = \frac{b'(1 + r) - \mathbb{E}[D(k', b', z')R(k', z') \mid z]}{b'\mathbb{E}[1 - D(k', b', z') \mid z]}. \quad (15)$$

This formula shows why the risky rate depends on default probabilities. If default becomes more likely, the denominator falls and the risky rate generally rises, all else equal. If $b' \leq 0$, the firm is not issuing positive debt in this simplified pricing equation; a separate cash-return convention should be specified.

Proposition 4 (The risky-debt model is a coupled fixed point). *A solution of the risky-debt model is a pair $(V^*, \tilde{r})$, together with optimal policies, satisfying all of the following statements at the same time:*

$$V^*(k, b, z) = \max\{0, G^*(k, b, z)\}, \quad (16)$$

$$D(k, b, z) = \mathbf{1}\{G^*(k, b, z) \leq 0\}, \quad (17)$$

$$b'(1 + r) = \mathbb{E} [D(k', b', z')R(k', z') + (1 - D(k', b', z'))b'(1 + \tilde{r}(z, k', b')) \mid z], \quad (18)$$

where G^ is computed from eq. (11) using the same V^* and $\tilde{r}$. The pricing equation is evaluated for each positive promised debt choice $b' > 0$, in particular for the policy choice made by the firm.*

Proof. Equation (16) is the limited-liability equity Bellman equation. Equation (17) says that default is chosen exactly when continuing gives shareholders no positive value. Equation (18) is the lender's zero-profit condition. These equations must hold together because each object uses the others: V^* determines the default set, the default set determines the risky rate through lender recovery, and the risky rate enters the cash flow used to compute G^* and therefore V^* . This channel operates both today and in future continuation values. Thus the problem is not a one-way recursion. It is a fixed point in which the equity value, default rule, optimal policy, and risky interest rate are solved jointly. $\square$

6 Writing default as equations

The expression $V = \max\{0, G\}$ is intuitive, but neural-network training often works better when a condition is written as equations. We can rewrite the max condition using a standard complementarity idea.

Proposition 5 (Limited liability as complementarity). *For two real numbers V and G , the statement*

$$V = \max\{0, G\}$$

is equivalent to the three conditions

$$V \geq 0, \quad V - G \geq 0, \quad V(V - G) = 0. \quad (19)$$

Proof. First suppose $V = \max\{0, G\}$. If $G \geq 0$, then $V = G$, so $V \geq 0$, $V - G = 0$, and $V(V - G) = 0$. If $G < 0$, then $V = 0$, so $V \geq 0$, $V - G = -G > 0$, and again $V(V - G) = 0$.

Now suppose the three conditions in eq. (19) hold. Since $V(V - G) = 0$, at least one of V and $V - G$ must be zero. If $V = 0$, then $V - G \geq 0$ implies $G \leq 0$, so $V = \max\{0, G\}$. If $V - G = 0$, then $V = G$, and $V \geq 0$, so $G \geq 0$ and again $V = \max\{0, G\}$. $\square$

One smooth way to combine the three complementarity conditions is the Fischer-Burmeister function

$$\text{FB}(a, c) = \sqrt{a^2 + c^2} - a - c. \quad (20)$$

It equals zero exactly when $a \geq 0$, $c \geq 0$, and $ac = 0$. Maliar et al. use this function for inequality constraints in their Bellman-equation example (Maliar et al., 2021, Section 4.5).

7 Neural-network approximations

Now approximate the unknown objects by functions with trainable parameters θ . The functions $\hat{h}_\theta^k$ and $\hat{h}_\theta^b$ approximate the optimal policy functions introduced in definition 1. The value network approximates V^* , not the value of an arbitrary initial policy. Write

$$\hat{V}_\theta(k, b, z) \approx V^*(k, b, z), \quad (21)$$

$$\hat{h}_\theta^k(k, b, z) \approx \text{optimal } k', \quad (22)$$

$$\hat{h}_\theta^b(k, b, z) \approx \text{optimal } b', \quad (23)$$

$$\hat{r}_\theta(z, k', b') \approx \tilde{r}(z, k', b'). \quad (24)$$

The hats remind us that these are approximations.

At a state $s = (k, b, z)$, define the network-implied controls

$$\hat{k}' = \hat{h}_\theta^k(k, b, z), \quad \hat{b}' = \hat{h}_\theta^b(k, b, z). \quad (25)$$

In an implementation, one often uses transformations to enforce constraints. For example, if capital must satisfy $\underline{k} \leq k' \leq \bar{k}$, one can set

$$\hat{h}_\theta^k(k, b, z) = \underline{k} + (\bar{k} - \underline{k})\sigma(N_\theta^k(k, b, z)),$$

where N_θ^k is an unrestricted neural network and $\sigma(x) = 1/(1 + e^{-x})$ maps every real number into $(0, 1)$.

8 Residuals for the risky-debt model

The model contains three main mathematical requirements:

1. optimal equity value must satisfy limited liability and the Bellman optimality equation;
2. the chosen policy $(\hat{k}', \hat{b}')$ must maximize the going-concern value;
3. the risky rate must satisfy the lender zero-profit condition.

We now turn each requirement into a residual. A residual is zero when the equation is satisfied and nonzero when the equation is violated.

8.1 Bellman and default residual

Define the network-implied optimal going-concern value at state $s = (k, b, z)$:

$$\begin{aligned} \hat{G}_\theta(k, b, z) = & e(k, \hat{k}', b, \hat{b}', z; \hat{r}_\theta) + \eta(e(k, \hat{k}', b, \hat{b}', z; \hat{r}_\theta)) \\ & + \beta \mathbb{E}[\hat{V}_\theta(\hat{k}', \hat{b}', z') \mid z]. \end{aligned} \quad (26)$$

Then the limited-liability Bellman residual is

$$\mathcal{B}_\theta(k, b, z) = \text{FB} \left(\hat{V}_\theta(k, b, z), \hat{V}_\theta(k, b, z) - \hat{G}_\theta(k, b, z) \right). \quad (27)$$

This residual is zero exactly when

$$\hat{V}_\theta(k, b, z) = \max\{0, \hat{G}_\theta(k, b, z)\}.$$

For a future state (k', b', z') , the expression $\hat{G}_\theta(k', b', z')$ means the same construction applied at that future state, using the network's candidate optimal policy choices for the period after that. It is the network analogue of G^* , not the value of a fixed initial policy h .

8.2 Debt-pricing residual

Default next period occurs when the next-period optimal equity value is zero:

$$V^*(k', b', z') = 0.$$

Using the optimal Bellman relation $V^* = \max\{0, G^*\}$, this is equivalent to $G^*(k', b', z') \leq 0$. Therefore the default indicator implied by the network is

$$\hat{D}_\theta(k', b', z') = \mathbf{1}\{\hat{G}_\theta(k', b', z') \leq 0\}. \quad (28)$$

This is an implementation choice based on the Section 3.6 default logic: the paper states that the firm defaults when equity value is zero, while the network uses the approximated going-concern value to classify that event. In the exact model, $V^* = 0$ and $G^* \leq 0$ are equivalent. For the approximating functions, this equivalence is exact only when the Bellman residual is zero, and otherwise it is an approximation whose quality should be checked. For numerical training, the sharp indicator can be replaced by a smooth approximation, for example

$$\hat{D}_{\theta, \kappa}(k', b', z') = \frac{1}{1 + \exp(\kappa \hat{G}_\theta(k', b', z'))},$$

where large $\kappa > 0$ makes the smooth function close to an indicator. With the sharp indicator, the pricing equation below represents the original default classification; with the smooth indicator, it represents a smoothed approximate model.

For positive promised debt $b' > 0$, define the pricing residual

$$\mathcal{P}_\theta(z, k', b') = b'(1 + r) - \mathbb{E} \left[\widehat{D}_\theta(k', b', z') R(k', z') + (1 - \widehat{D}_\theta(k', b', z')) b' (1 + \widehat{r}_\theta(z, k', b')) \mid z \right]. \quad (29)$$

The residual $\mathcal{P}_\theta$ is zero when the lender breaks even. For fixed current z and chosen (k', b') , the expectation integrates over future shocks z' , because those shocks determine whether the future state (k', b', z') is a default state or a solvent state.

8.3 Policy maximization residual

To keep the lecture self-contained, we use a direct maximization residual rather than first-order conditions. The idea is simple: the chosen action should do at least as well as any alternative action.

For any candidate alternative $a = (\bar{k}', \bar{b}')$, define

$$\widehat{Q}_\theta(k, b, z; a)$$

by taking the right side of eq. (10), replacing V and $\tilde{r}$ by their neural-network approximations, and using the action a . The policy loss against the alternative a is

$$\mathcal{M}_\theta(k, b, z; a) = \max \left\{ 0, \widehat{Q}_\theta(k, b, z; a) - \widehat{Q}_\theta(k, b, z; (\widehat{k}', \widehat{b}')) \right\}. \quad (30)$$

Proposition 6 (Direct policy residual). *Fix a state (k, b, z) . If*

$$\mathcal{M}_\theta(k, b, z; a) = 0$$

for every feasible alternative action $a = (\bar{k}', \bar{b}')$, then the network-chosen action $(\widehat{k}', \widehat{b}')$ maximizes $\widehat{Q}_\theta(k, b, z; \cdot)$ over the feasible action set.

Proof. The equation $\mathcal{M}_\theta(k, b, z; a) = 0$ means

$$\widehat{Q}_\theta(k, b, z; a) - \widehat{Q}_\theta(k, b, z; (\widehat{k}', \widehat{b}')) \leq 0$$

for that alternative action a . Therefore

$$\widehat{Q}_\theta(k, b, z; (\widehat{k}', \widehat{b}')) \geq \widehat{Q}_\theta(k, b, z; a).$$

If this holds for every feasible a , then no feasible alternative gives a higher value. That is exactly the definition of a maximizer. $\square$

Remark 4 (Sampled alternatives are not global proof). *The proposition is an exact mathematical statement because it assumes every feasible alternative action is checked. In computation, we usually test only a finite sample of alternatives. Zero residuals on that finite test set certify only that the chosen policy beats the sampled alternatives, not that it is a global maximizer over all possible (k', b') .*

8.4 Euler residuals from first-order conditions

Directly minimizing Bellman residuals can be numerically difficult. This is one reason Euler-equation methods are common in dynamic economics and reinforcement learning. Maliar et al. also describe Euler-residual minimization (Maliar et al., 2021, Definitions 2.6 and 2.7). In the risky-debt model, Euler residuals are useful, but only in the smooth part of the problem: the firm must be continuing, the policy choice must be interior, and the risky rate and value function must be differentiable. At default boundaries, borrowing limits, or other kinks, one needs complementarity conditions, subgradients, or smoothing.

Let

$$m(e) = 1 + \eta'(e)$$

be the marginal value of one more dollar of current cash flow. If external finance is costly, then $m(e)$ is high when the firm is short of cash. For an interior continuation state, define

$$\bar{e} = e(k, k', b, b', z; \tilde{r}(z, k', b')).$$

The total derivative of current cash flow with respect to a choice must include the effect of that choice on the risky rate:

$$\frac{d\bar{e}}{dk'} = -(1 + \psi_I(I, k)) + e_{\tilde{r}} \frac{\partial \tilde{r}(z, k', b')}{\partial k'}, \quad (31)$$

$$\frac{d\bar{e}}{db'} = \frac{1}{1 + \tilde{r}} + \frac{\tau \tilde{r}}{(1 + \tilde{r})(1 + r)} + e_{\tilde{r}} \frac{\partial \tilde{r}(z, k', b')}{\partial b'}, \quad (32)$$

where $I = k' - (1 - \delta)k$, ψ_I is the derivative of ψ with respect to investment, and

$$e_{\tilde{r}} = \frac{b'}{(1 + \tilde{r})^2} \left(-1 + \frac{\tau}{1 + r} \right). \quad (33)$$

All terms in eqs. (31) to (33) are evaluated at (k, k', b, b', z) .

Proposition 7 (Interior first-order conditions). *Suppose the current state is a continuation state, the optimal action (k', b') is interior, and the relevant functions are differentiable. Then the optimal policy satisfies*

$$0 = m(\bar{e}) \frac{d\bar{e}}{dk'} + \beta \mathbb{E}[V_k^*(k', b', z') \mid z], \quad (34)$$

$$0 = m(\bar{e}) \frac{d\bar{e}}{db'} + \beta \mathbb{E}[V_b^*(k', b', z') \mid z]. \quad (35)$$

Proof. At an interior continuation state, the firm chooses (k', b') to maximize

$$e + \eta(e) + \beta \mathbb{E}[V^*(k', b', z') \mid z].$$

Differentiate this expression with respect to k' and b' . The derivative of $e + \eta(e)$ is $(1 + \eta'(e))de = m(e)de$. The derivative of the continuation term is the expected derivative of V^* with respect to the next state. Setting both derivatives equal to zero gives eqs. (34) and (35). $\square$

The first-order conditions still contain derivatives of the value function. Those derivatives can be obtained by automatic differentiation of a value network. Alternatively, in smooth continuation states, we can use envelope conditions to eliminate them.

Proposition 8 (Envelope derivatives). *Away from the default boundary, write*

$$h^{k\star} = h^{k\star}(k, b, z), \quad h^{b\star} = h^{b\star}(k, b, z),$$

and define

$$A(k, b, z) = (1 - \tau)\pi_k(k, z) + (1 - \delta)(1 + \psi_I(I, k)) - \psi_k(I, k),$$

where $I = h^{k\star}(k, b, z) - (1 - \delta)k$. Then

$$V_k^\star(k, b, z) = (1 - D(k, b, z)) m(e(k, h^{k\star}, b, h^{b\star}, z; \tilde{r})) A(k, b, z), \quad (36)$$

$$V_b^\star(k, b, z) = -(1 - D(k, b, z)) m(e(k, h^{k\star}, b, h^{b\star}, z; \tilde{r})). \quad (37)$$

Proof. In a default state, $V^\star = 0$ locally, so the derivative is zero away from the boundary. In a continuation state, $V^\star = G^\star$. The envelope theorem says that when differentiating with respect to the current state variables (k, b) , we can hold the optimal choices fixed. Differentiating current cash flow with respect to k gives $A(k, b, z)$, and differentiating with respect to beginning-of-period debt b gives -1 . Multiplying by the marginal value of cash flow $m(e)$ gives eqs. (36) and (37). $\square$

Combining the first-order conditions and the envelope derivatives gives an Euler-residual objective. For a network policy, let

$$\hat{\mathcal{E}}_\theta^k(k, b, z) = \hat{m}_t \frac{d\hat{e}_t}{dk'} + \beta \mathbb{E}[(1 - \hat{D}_{\theta, t+1}) \hat{m}_{t+1} \hat{A}_{t+1} \mid z], \quad (38)$$

and

$$\hat{\mathcal{E}}_\theta^b(k, b, z) = \hat{m}_t \frac{d\hat{e}_t}{db'} - \beta \mathbb{E}[(1 - \hat{D}_{\theta, t+1}) \hat{m}_{t+1} \mid z]. \quad (39)$$

Here $\hat{e}_t$ is current cash flow under the network policy, $\hat{m}_t = 1 + \eta'(\hat{e}_t)$, and the $t + 1$ quantities are computed at the future state $(\hat{k}', \hat{b}', z')$ using the network's next policy choices. The derivative terms $d\hat{e}_t/dk'$ and $d\hat{e}_t/db'$ include the derivative of the risky rate network with respect to the action. In practice, automatic differentiation can compute these terms.

An Euler-style loss can replace the direct policy maximization loss on states where the policy is smooth and interior:

$$\mathcal{L}^{\text{Euler}}(\theta) = \mathbb{E}_{(k, b, z) \sim \mu} \left[w_E^k (\hat{\mathcal{E}}_\theta^k)^2 + w_E^b (\hat{\mathcal{E}}_\theta^b)^2 + w_P \mathcal{P}_\theta^2 + w_B \mathcal{B}_\theta^2 \right]. \quad (40)$$

The Bellman/default residual $\mathcal{B}_\theta$ is still useful because Euler equations do not by themselves locate the default boundary or pin down the level of equity value. In applications one may put most of the weight on the Euler and pricing residuals in smooth continuation regions, and use $\mathcal{B}_\theta$ mainly to discipline limited liability and default. Thus eq. (40) should be read as the interior part of the method, not as the full risky-debt training objective.

8.5 Training with Euler residuals and nonsmooth regions

The practical solver should not train Euler residuals everywhere. Instead, it should use different residuals in different regions of the state space. Let

$$C_\theta(k, b, z) = \mathbf{1}\{\hat{G}_\theta(k, b, z) > 0\}$$

be the network's continuation indicator, and let

$$I_\theta(k, b, z)$$

be an interior indicator that equals one when the policy choices are safely away from imposed numerical bounds, such as $\underline{k} < \widehat{k}' < \bar{k}$ and $\underline{b} < \widehat{b}' < \bar{b}$. The Euler residual should receive full weight only when

$$S_\theta(k, b, z) = C_\theta(k, b, z)I_\theta(k, b, z) = 1.$$

In code, the sharp indicators above are often replaced by smooth gates. For example, a smooth continuation gate can be

$$C_{\theta, \kappa}(k, b, z) = \frac{1}{1 + \exp(-\kappa \widehat{G}_\theta(k, b, z))},$$

where large κ makes the gate close to the sharp continuation indicator.

If the firm continues but the action is not interior, the first-order condition $0 = \partial Q / \partial a$ should be replaced by a Karush-Kuhn-Tucker condition. If the firm defaults, the action FOC is not the right condition, because shareholders are choosing the outside option with value zero. That region is disciplined by the limited-liability Bellman residual.

For a scalar choice $x \in [\underline{x}, \bar{x}]$, where x stands for either k' or b' , let

$$g_x(k, b, z) = \left. \frac{\partial \widehat{Q}_\theta(k, b, z; k', b')}{\partial x} \right|_{(k', b') = (\widehat{k}', \widehat{b}')}.$$

be the network-implied derivative with respect to that choice, holding the other choice fixed at its network value. Here g_x is the derivative of the objective being maximized, not the derivative of the negative objective. At a maximum, the derivative must satisfy

$$g_x \leq 0 \quad \text{at } x = \underline{x}, \quad g_x = 0 \quad \text{inside the interval}, \quad g_x \geq 0 \quad \text{at } x = \bar{x}.$$

Equivalently, with lower and upper multipliers $\lambda_x^L, \lambda_x^U \geq 0$,

$$0 = g_x + \lambda_x^L - \lambda_x^U, \tag{41}$$

$$0 = \lambda_x^L(x - \underline{x}), \tag{42}$$

$$0 = \lambda_x^U(\bar{x} - x). \tag{43}$$

These equations can be implemented by adding multiplier networks and using Fischer-Burmeister residuals for the lower and upper bounds:

$$\begin{aligned} \mathcal{K}_\theta^x(k, b, z) &= (g_x + \lambda_x^L - \lambda_x^U)^2 \\ &\quad + \text{FB}(\lambda_x^L, x - \underline{x})^2 + \text{FB}(\lambda_x^U, \bar{x} - x)^2. \end{aligned} \tag{44}$$

The full action-bound residual is

$$\mathcal{K}_\theta = \mathcal{K}_\theta^{k'} + \mathcal{K}_\theta^{b'},$$

with each term evaluated at the network action. This is the same complementarity idea used earlier for limited liability.

The training idea is now simple. At a safe interior continuation state, a good policy should make the Euler residuals small. At a continuation state on an action boundary, a good policy should make the KKT residual small instead. At a default state, the policy action is economically irrelevant to shareholders because they choose the outside option, so the Bellman/default residual is the main condition. The debt-pricing residual is separate: whenever the chosen debt is positive, lenders must break even under the default probabilities implied by the same value and policy networks.

The hybrid Euler-KKT loss is therefore

$$\begin{aligned} \mathcal{L}^{\text{hybrid}}(\theta) = \mathbb{E}_{(k,b,z) \sim \mu} & \left[w_E S_\theta((\hat{\mathcal{E}}_\theta^k)^2 + (\hat{\mathcal{E}}_\theta^b)^2) \right. \\ & \left. + w_K C_\theta(1 - I_\theta) \mathcal{K}_\theta(k, b, z) + w_P \mathcal{P}_\theta(z, \hat{k}', \hat{b}')^2 + w_B \mathcal{B}_\theta(k, b, z)^2 \right], \end{aligned} \quad (45)$$

where the KKT term is weighted only in continuation states. More explicitly, if

$$J_\theta(k, b, z) = \mathbf{1}\{\hat{b}'_\theta(k, b, z) > 0\}$$

is a positive-debt gate, then many implementations use

$$w_P J_\theta \mathcal{P}_\theta(z, \hat{k}', \hat{b}')^2$$

in place of the ungated pricing term. If $b' \leq 0$, the pricing term is either omitted or replaced by the chosen cash-return convention; for positive debt choices it is always enforced. If the implementation uses action transformations that always keep $\hat{k}'$ and $\hat{b}'$ strictly inside their bounds, then $\mathcal{K}_\theta$ may be unnecessary, but the Euler residual can become weak near the artificial boundary because the transformation saturates.

Proposition 9 (What the hybrid residuals certify). *Suppose the Bellman/default residual, pricing residual, Euler residuals on smooth interior continuation states, and KKT residuals on constrained states are all zero on a region. Then the approximating functions satisfy the local necessary optimality conditions, limited liability, and debt-pricing conditions on that region.*

Proof. On smooth interior continuation states, zero Euler residuals are exactly the first-order conditions from proposition 7. On constrained states, zero KKT residuals are the first-order necessary conditions with active bounds. Zero Bellman/default residual enforces the limited-liability default condition, and zero pricing residual enforces lender break-even. Together these conditions are the local necessary conditions for the risky-debt problem. $\square$

Remark 5 (Euler residuals are not a global optimality certificate). *Euler and KKT residuals are local conditions. They can miss a better distant action when the objective is nonconcave. For that reason, a practical solver should still test the trained policy against random alternative actions, report out-of-sample Bellman and pricing residuals, and simulate the model to check that default and leverage behavior are economically sensible.*

8.6 Hard constraints as output transformations

The KKT residuals above are one way to handle action bounds. A different idea, emphasized by Hall-Hoffarth (2023), is to give constraints *lexicographic priority*: first restrict the neural network so that its outputs are feasible by construction, and only then train the remaining optimality residuals. In the language of the paper, these are *hard constraints*, in contrast to *soft constraints* that are enforced only through penalty terms such as Fischer-Burmeister losses.

The simplest examples are scalar bounds. If the raw neural network output is an unrestricted number $n_k(k, b, z)$, define

$$\hat{k}' = \underline{k} + (\bar{k} - \underline{k})\sigma(n_k(k, b, z)), \quad \sigma(x) = \frac{1}{1 + \exp(-x)}. \quad (46)$$

Then $\underline{k} < \hat{k}' < \bar{k}$ automatically. Similarly,

$$\hat{b}' = \underline{b} + (\bar{b} - \underline{b})\sigma(n_b(k, b, z)) \quad (47)$$

automatically satisfies $\underline{b} < \widehat{b}' < \bar{b}$. If a variable only needs to be positive, a softplus transformation can be used:

$$x = \epsilon + \log(1 + \exp(n_x)), \quad \epsilon > 0. \quad (48)$$

For example, one may parameterize the risky gross rate as

$$1 + \widehat{r}_\theta(z, k', b') = \epsilon_r + \log(1 + \exp(n_r(z, k', b'))),$$

which ensures a positive gross promised return.

The smooth sigmoid transformation is best interpreted as a way to impose artificial numerical bounds. If a bound is a genuine economic constraint that may bind in equilibrium, a strictly interior sigmoid may be too restrictive because it cannot place the choice exactly on the boundary. In that case one can use a transformation that allows binding, such as a clipped positive output, or keep the KKT/complementarity residual for that economic constraint. Clipping can create nondifferentiable or flat-gradient regions, so it is a practical trade-off rather than a free improvement. This is the same reason [Hall-Hoffarth \(2023\)](#) distinguishes hard feasibility from ordinary penalty-based constraint handling.

Proposition 10 (Hard numerical bounds remove numerical-bound KKT residuals). *Suppose the policy networks are parameterized by eqs. (46) and (47). Then the numerical bounds on k' and b' are satisfied at every state and every parameter value. If the transformation stays strictly inside artificial numerical bounds that are not intended to bind as economic constraints, the usual interior Euler conditions replace the numerical-bound KKT residuals on those controls.*

Proof. The sigmoid function always lies between zero and one. Therefore $\underline{k} + (\bar{k} - \underline{k})\sigma(n_k)$ always lies strictly between $\underline{k}$ and $\bar{k}$, and the same argument applies to b' . Since the transformed choices are interior points of the numerical interval, no lower or upper numerical bound is active. Thus, for those artificial box bounds, there is no active bound multiplier to learn. The relevant local optimality condition is the Euler first-order condition with respect to the free raw network output, equivalently the chain-rule version of the interior condition with respect to the transformed control when the transformation has a nonzero derivative. Numerically, a saturating sigmoid can still make gradients very small near the artificial boundary. $\square$

This hard-constraint idea is useful whenever part of a model can be solved by algebra. For example, if an accounting identity gives one variable as an exact function of the others, it is usually better to build that identity into the network output than to ask the network to rediscover it from a penalty term. The principle is: do not ask the network to learn equations that can be satisfied exactly by construction.

For the risky-debt model, hard constraints are useful but not sufficient. They can enforce:

- numerical bounds on k' and b' ;
- positivity of quantities such as gross risky returns;
- accounting identities or algebraic definitions that can be solved explicitly from other variables.

They cannot, by themselves, solve the default free-boundary condition

$$V(k, b, z) = \max\{0, G(k, b, z)\}.$$

The reason is that default is not merely a bound on a single output. It is a regime choice: in continuation states equity value equals going-concern value, while in default states equity value equals

zero. A transformation such as $\widehat{V}_\theta = \text{softplus}(n_V)$ guarantees $\widehat{V}_\theta \geq 0$, but it does not guarantee $\widehat{V}_\theta = \widehat{G}_\theta$ in continuation states or $\widehat{V}_\theta = 0$ in default states. Therefore the Bellman/default residual $\mathcal{B}_\theta$ remains necessary. Even if one hard-codes $\widehat{V}_\theta = \max\{0, \widehat{G}_\theta\}$ with a smooth approximation, one still has to learn the correct continuation object $\widehat{G}_\theta$ and the default boundary consistently with debt pricing.

The recommended practical objective is therefore a hard-constrained hybrid loss:

$$\begin{aligned} \mathcal{L}^{\text{hard-hybrid}}(\theta) = \mathbb{E}_{(k,b,z) \sim \mu} \Big[& w_E C_\theta((\widehat{\mathcal{E}}_\theta^k)^2 + (\widehat{\mathcal{E}}_\theta^b)^2) \\ & + w_P J_\theta \mathcal{P}_\theta(z, \widehat{k}', \widehat{b}')^2 + w_B \mathcal{B}_\theta(k, b, z)^2 \Big], \end{aligned} \quad (49)$$

where the action-bound KKT term is omitted because the relevant bounds are already enforced by the output layer. If some genuine economic inequality is not hard-coded, or if the implementation uses clipping that creates flat-gradient regions, then the corresponding KKT or complementarity residual should be kept.

Remark 6 (Hard constraints do not prove optimality). *Hard constraints guarantee feasibility, not optimality. They make it impossible for the network to violate selected constraints, which can greatly improve training stability. The Euler, pricing, Bellman/default, and validation diagnostics are still needed to check whether the feasible policy is close to an economic solution.*

9 The training objective

Let μ be a distribution over states (k, b, z) . It may be a fixed training distribution over a chosen rectangle, or it may be the simulated ergodic distribution generated by the current policy, as in the stochastic simulation logic emphasized by [Maliar et al. \(2021, Section 3\)](#).

There are now two natural training choices. The first uses sampled alternative actions to test optimality directly. Let $\nu(\cdot \mid k, b, z)$ be a distribution over alternative actions used to test whether the policy is close to optimal. The theoretical risk is

$$\begin{aligned} \mathcal{L}(\theta) = \mathbb{E}_{(k,b,z) \sim \mu} \Big[& w_B \mathcal{B}_\theta(k, b, z)^2 + w_P \mathcal{P}_\theta(z, \widehat{k}', \widehat{b}')^2 \\ & + w_M \mathbb{E}_{a \sim \nu(\cdot \mid k, b, z)} [\mathcal{M}_\theta(k, b, z; a)^2] \Big], \end{aligned} \quad (50)$$

where $w_B, w_P, w_M > 0$ are weights chosen so that the different residuals have comparable numerical scale. This is called a *weighted scalarization*: many requirements are collapsed into one number before the optimizer sees them. It is simple, but the choice of weights can be difficult. The pricing residual, Bellman/default residual, Euler residuals, and direct maximization residuals can have different units, different numerical sizes, and different training speeds.

The second choice uses the hybrid Euler-KKT objective in eq. (45). This is often cheaper in high-dimensional action spaces because it avoids searching over many alternative actions. However, it gives local first-order evidence, not global maximization evidence. If the policy outputs are hard-constrained as in section 8.6, the hard-constrained hybrid objective in eq. (49) can be used instead.

The empirical training loss with N sampled states for the direct maximization version is

$$\mathcal{L}_N(\theta) = \frac{1}{N} \sum_{i=1}^N \left[w_B \mathcal{B}_\theta(s_i)^2 + w_P \mathcal{P}_\theta(z_i, \widehat{k}'_i, \widehat{b}'_i)^2 + w_M \mathcal{M}_\theta(s_i; a_i)^2 \right]. \quad (51)$$

The empirical hybrid Euler-KKT loss replaces the policy-maximization term by Euler and KKT terms:

$$\begin{aligned}\mathcal{L}_N^{\text{hybrid}}(\theta) = \frac{1}{N} \sum_{i=1}^N & \left[w_E S_\theta(s_i) ((\hat{\mathcal{E}}_\theta^k(s_i))^2 + (\hat{\mathcal{E}}_\theta^b(s_i))^2) \right. \\ & + w_K C_\theta(s_i) (1 - I_\theta(s_i)) \mathcal{K}_\theta(s_i) + w_P J_\theta(s_i) \mathcal{P}_\theta(z_i, \hat{k}'_i, \hat{b}'_i)^2 \\ & \left. + w_B \mathcal{B}_\theta(s_i)^2 \right].\end{aligned}\quad (52)$$

Training means approximately solving either

$$\theta^\star \in \arg \min_{\theta} \mathcal{L}_N(\theta). \quad (53)$$

or the same minimization with $\mathcal{L}_N^{\text{hybrid}}$ in place of $\mathcal{L}_N$. If hard output constraints are used, the empirical analogue of eq. (49) is

$$\begin{aligned}\mathcal{L}_N^{\text{hard-hybrid}}(\theta) = \frac{1}{N} \sum_{i=1}^N & \left[w_E C_\theta(s_i) ((\hat{\mathcal{E}}_\theta^k(s_i))^2 + (\hat{\mathcal{E}}_\theta^b(s_i))^2) \right. \\ & \left. + w_P J_\theta(s_i) \mathcal{P}_\theta(z_i, \hat{k}'_i, \hat{b}'_i)^2 + w_B \mathcal{B}_\theta(s_i)^2 \right].\end{aligned}\quad (54)$$

9.1 Replacing fixed weights by multi-objective training

The fixed weights in eqs. (50), (52) and (54) are often the least pleasant part of the computation. If w_P is too small, the equity policy may look good while the debt price violates lender break-even. If w_B is too small, the default boundary may drift. If w_E is too large, the solver may chase local Euler equations while ignoring the level of the value function. There is no universal scale that makes these choices automatic.

A practical alternative is *multi-objective optimization* (MOO). Instead of relying only on one permanent weighted sum, we keep the residual losses separate and let the current gradients help decide the update direction. This is the viewpoint of the multiple-gradient descent algorithm of Désidéri (2012); Sener and Koltun (2018) use the same broad multi-objective perspective for neural-network multi-task learning. For example, with hard output constraints one may define the mini-batch objectives

$$\mathcal{L}_E(\theta) = \frac{1}{N} \sum_{i=1}^N C_\theta(s_i) \left[(\hat{\mathcal{E}}_\theta^k(s_i))^2 + (\hat{\mathcal{E}}_\theta^b(s_i))^2 \right], \quad (55)$$

$$\mathcal{L}_P(\theta) = \frac{1}{N} \sum_{i=1}^N J_\theta(s_i) \mathcal{P}_\theta(z_i, \hat{k}'_i, \hat{b}'_i)^2, \quad (56)$$

$$\mathcal{L}_B(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{B}_\theta(s_i)^2. \quad (57)$$

If action-bound KKT terms, direct policy-check terms, or local-anchor terms are needed, each can be added as another objective. Thus the optimizer sees a vector

$$\mathcal{L}(\theta) = (\mathcal{L}_1(\theta), \dots, \mathcal{L}_m(\theta)),$$

not just the scalar sum $\sum_{j=1}^m w_j \mathcal{L}_j(\theta)$. For gradient-based training, the gates C_θ and J_θ are usually implemented as smooth approximations, or the derivative statement below is read as applying away from the default and zero-debt kinks.

The intuition is simple. Each residual loss pulls the network parameters in a direction. Fixed weights decide in advance which pull is louder. MOO computes the pulls on the current mini-batch and combines them adaptively.

Definition 3 (Common descent and Pareto stationarity). *Let $g_j(\theta) = \nabla_\theta \mathcal{L}_j(\theta)$ be the gradient of the j -th objective. A direction p is a common descent direction if*

$$g_j(\theta)^\top p < 0 \quad \text{for every } j.$$

This means that a small move from θ in direction p lowers every objective to first order. A point is first-order Pareto stationary if no common descent direction exists.

The easiest MOO rule to write down is the minimum-norm gradient rule used in MGDA (Désidéri, 2012). Let

$$\Delta_m = \left\{ \alpha \in \mathbb{R}^m : \alpha_j \geq 0, \quad \sum_{j=1}^m \alpha_j = 1 \right\}$$

be the probability simplex. At a training step, compute all objective gradients

$$g_1, \dots, g_m.$$

MGDA chooses

$$\alpha^\star \in \arg \min_{\alpha \in \Delta_m} \left\| \sum_{j=1}^m \alpha_j g_j \right\|^2, \quad d^\star = \sum_{j=1}^m \alpha_j^\star g_j. \quad (58)$$

Then the neural-network optimizer uses $d^\star$ as the combined gradient, so the parameter update is roughly

$$\theta_{\text{new}} = \theta - \gamma d^\star$$

for a learning rate $\gamma > 0$, possibly after Adam or another stochastic optimizer rescales the coordinates.

Proposition 11 (MGDA gives a balanced first-order update). *Suppose the objectives are differentiable at the current parameter vector θ , and let $d^\star$ be defined by eq. (58). If $d^\star = 0$, then θ is first-order Pareto stationary. If $d^\star \neq 0$, then $-d^\star$ is a common descent direction:*

$$g_j^\top (-d^\star) \leq -\|d^\star\|^2 < 0 \quad \text{for every } j. \quad (59)$$

Proof. The vector $d^\star$ is the point in the convex hull of $\{g_1, \dots, g_m\}$ that is closest to zero. Take any other point y in this convex hull. For $0 < t < 1$, the point

$$d_t = (1-t)d^\star + ty$$

is also in the convex hull. Since $d^\star$ is the closest point to zero, $\|d_t\|^2 \geq \|d^\star\|^2$. Expanding this inequality gives

$$2t(d^\star)^\top (y - d^\star) + t^2\|y - d^\star\|^2 \geq 0.$$

Divide by t and let t become very small. We get

$$(d^\star)^\top y \geq \|d^\star\|^2.$$

Each gradient g_j is itself a point in the convex hull, so

$$g_j^\top d^\star \geq \|d^\star\|^2.$$

If $d^\star \neq 0$, multiplying by -1 gives eq. (59).

Now suppose $d^\star = 0$. Then zero is a convex combination of the gradients:

$$0 = \sum_{j=1}^m \alpha_j^\star g_j.$$

If a common descent direction p existed, then $g_j^\top p < 0$ for every j , and therefore

$$0 = \left(\sum_{j=1}^m \alpha_j^\star g_j \right)^\top p = \sum_{j=1}^m \alpha_j^\star g_j^\top p < 0,$$

which is impossible. Hence no common descent direction exists. $\square$

This proposition is a first-order statement. It says what happens for an infinitesimally small step when the objectives are smooth. It does not say that a finite Adam step always lowers every loss on every mini-batch, and it does not prove that a neural network has solved the economic model. It does, however, explain why MOO is attractive: it reduces reliance on hand-tuned fixed weights by using an adaptive combination based on the current gradient conflict.

A practical MGDA training step is as follows. First form one loss for each economic condition. Then compute one gradient vector for each loss and build the Gram matrix

$$G_{ij} = g_i^\top g_j,$$

solve eq. (58) for the simplex weights $\alpha^\star$, and update the network using the combined gradient $d^\star$. The Gram matrix is useful because it lets us compute the MGDA objective

$$\left\| \sum_{j=1}^m \alpha_j g_j \right\|^2 = \alpha^\top G \alpha$$

without writing out every coordinate of every gradient in the lecture note. During training one should record diagnostics such as each objective loss, heldout objective losses, gradient norms, the gradient cosine matrix, the simplex weights $\alpha_j^\star$, and the directional derivatives $g_j^\top d^\star$. These diagnostics matter because MOO reduces manual weight tuning; it does not remove the need to monitor whether every economic equation is improving. In practice, MOO should be read as reducing sensitivity to weights, not abolishing all scale and normalization choices.

For the risky-debt solver, the practical recipe is:

1. Use hard output transformations for simple feasibility requirements whenever possible.
2. Split the remaining conditions into separate objectives: Euler residuals, debt-pricing residuals, Bellman/default residuals, KKT or complementarity residuals when needed, and direct policy-check or local-anchor residuals when global policy errors are a concern.
3. Choose economically meaningful scales for the objective components. MOO reduces the need for fixed weights, but it does not magically fix bad units. For example, an Euler residual measured in marginal values and a pricing residual measured in promised debt payments should not be compared blindly without considering their natural economic scales.

4. On each mini-batch, compute one gradient g_j for each objective and aggregate the gradients by the MGDA rule in eq. (58).
5. Keep per-objective validation diagnostics. A small aggregate update is not enough; pricing, default, Euler, and action-check residuals should all be reported separately.
6. Use economic veto rules when selecting a final network. For example, if the pricing residual or Bellman/default residual deteriorates beyond an acceptable tolerance on heldout states, reject the network even if an average scalar loss looks good.

This makes the method more practical because the researcher no longer has to guess a single permanent set of weights for all residuals. The remaining choices are more interpretable: how to group the objectives, how to normalize them, what heldout diagnostics must pass, and which MOO aggregation rule is stable for the problem size.

Proposition 12 (Zero direct residuals imply a solved approximate model). *Suppose the residuals*

$$\mathcal{B}_\theta, \quad \mathcal{P}_\theta, \quad \mathcal{M}_\theta$$

are all zero for every state in the training region and for every feasible alternative action used in the maximization test. Then the approximating functions $\hat{V}_\theta$, $\hat{h}_\theta^k$, $\hat{h}_\theta^b$, and $\hat{r}_\theta$ satisfy the risky-debt Bellman equation, the lender zero-profit condition, and the policy optimality condition on that region.

Proof. If $\mathcal{B}_\theta = 0$, then by the Fischer-Burmeister equivalence the approximated value function satisfies

$$\hat{V}_\theta(k, b, z) = \max\{0, \hat{G}_\theta(k, b, z)\}.$$

This is the limited-liability Bellman equation for the approximating functions.

If $\mathcal{P}_\theta = 0$, then the expected payoff to the lender equals $b'(1+r)$. This is exactly the zero-profit debt-pricing condition.

If $\mathcal{M}_\theta = 0$ for every feasible alternative action, then proposition 6 says that the network policy maximizes the network-implied going-concern value. Combining the three statements gives the claim. $\square$

10 Comparison with Duarte and Fonseca’s structural-estimation algorithm

A recent paper by Duarte and Fonseca (2026) applies a closely related neural idea to structural estimation. Their application is a dynamic corporate finance model with leverage, investment, cash, endogenous default, costly equity issuance, and non-convex adjustment costs. The paper is especially useful for us because it studies a model with many of the same computational difficulties as the risky-debt problem in this note: a value function, policy functions, default, risky debt prices, and nonsmooth payoffs. Since the paper is a recent working paper, we use it here as a carefully cited algorithmic reference, not as a theorem that all such neural estimators must converge.

10.1 What their algorithm does

The algorithm in Duarte and Fonseca (2026, Section 3) has three main blocks.

First, it solves the firm’s dynamic problem by neural policy iteration. The value network is written as a function not only of the economic state, but also of the structural parameters:

$$V(z, k, b, \beta; \Theta_V),$$

where β denotes model parameters and Θ_V denotes neural-network weights. This is sometimes called a *universal value function*: one network represents many parameter-specific value functions. For a fixed policy, the value-network weights are trained to reduce a squared Bellman residual (Duarte and Fonseca, 2026, Section 3.1.1 and eqs. (14)–(15)). Then, holding the value network fixed, policy networks are trained to increase the right-hand side of the Bellman equation (Duarte and Fonseca, 2026, Section 3.1.2 and eqs. (16)–(17)). The algorithm alternates between these two steps.

Second, the trained value and policy networks are used to simulate firm panels for many parameter vectors. In their application, the authors refine the neural solution on a grid before simulation. The refinement uses exact nonsmoothed payoffs, local policy improvement, policy evaluation, and recomputation of bond prices (Duarte and Fonseca, 2026, Section 3.2.1 and Appendix A.5).

Third, the paper trains separate *moment networks*. If m_j is the j -th simulated moment, the moment network approximates

$$g_j(\beta) = \mathbb{E}[m_j \mid \beta].$$

These networks turn simulated model moments into differentiable functions of the parameters. Estimation then minimizes the usual weighted distance between data moments and model moments,

$$\hat{\beta} \in \arg \min_{\beta} (\hat{m} - g(\beta))^{\top} W (\hat{m} - g(\beta)), \quad (60)$$

using gradients of the moment networks (Duarte and Fonseca, 2026, Section 3.3 and eqs. (18)–(21)). For a general equilibrium extension, they also treat the wage as a pseudo parameter and add a market-clearing imbalance as another target moment (Duarte and Fonseca, 2026, Section 3.4). Finally, they use minimum-loss functions to shrink parameter bounds and diagnose identification (Duarte and Fonseca, 2026, Sections 3.5–3.6).

10.2 How this compares with the first approach in this note

The first approach in this note is the direct value-and-policy approach: learn

$$\hat{V}_{\theta}, \quad \hat{h}_{\theta}^k, \quad \hat{h}_{\theta}^b, \quad \hat{r}_{\theta}$$

by minimizing Bellman/default, debt-pricing, and policy-maximization residuals. At the level of the *model solution block*, this is very close in spirit to the first block of Duarte and Fonseca (2026). Both methods use neural networks for value and policy functions. Both train with stochastic gradients and automatic differentiation. Both can sample states instead of storing the value function on a full grid. Both can smooth nonsmooth pieces during training and then check accuracy with more direct policy or grid diagnostics.

So your intuition is essentially right, with one important qualification. If our only goal is to solve one risky-debt model at one fixed parameter vector, if we use the direct Bellman/policy objective rather than Euler residuals, and if both methods impose the same lender zero-profit pricing equation, then the solution step is not fundamentally different from the value/policy part of Duarte and Fonseca (2026). The difference is mostly numerical organization: this note writes the Bellman/default, pricing, and policy conditions as residuals to be minimized jointly, while Duarte and Fonseca (2026) present an alternating policy-evaluation and policy-improvement procedure, with a lagged bond-price target during training and a grid-refinement check afterward.

However, the full Duarte–Fonseca algorithm is more than a solver. It is a structural-estimation pipeline. It solves across many parameter values at once, simulates moments, trains moment-surrogate networks, estimates parameters using eq. (60), treats general-equilibrium prices as pseudo parameters when needed, and uses minimum-loss functions for global identification diagnostics.

Those parts are not contained in the basic Maliar–Maliar–Winant solution method by themselves. They are built on top of a neural solution method.

Proposition 13 (Solver equivalence, but not estimation equivalence). *For a fixed parameter vector and the same lender zero-profit pricing equation, a direct Bellman/policy neural solver of the kind described in this note and the value/policy block of Duarte and Fonseca (2026) attack the same mathematical fixed point: value, policy, default, and risky debt prices must be mutually consistent. They differ mainly in training organization. But the full Duarte–Fonseca structural-estimation algorithm contains additional moment, estimation, market-clearing, and identification steps that are not implied by solving the Bellman equation alone.*

Proof. Both solver blocks try to find a value function and policy functions satisfying the same type of Bellman optimality condition: the value equals current payoff plus discounted expected continuation value under optimal controls, with limited liability/default handled through a max or default rule. In a risky debt model, the debt price must also satisfy the lender’s zero-profit pricing equation, because that price enters the firm’s payoff and is itself determined by future default. A direct residual method minimizes violations of these conditions and violations of policy optimality. Neural policy iteration alternates between fitting the value under the current policy and improving the policy using the fitted value. If both procedures converge exactly and use the same debt-pricing condition, they satisfy the same Bellman, pricing, and policy optimality equations.

The moment-network and estimation steps are different objects. They take a solved model, simulate model-implied moments, approximate the parameter-to-moment map, and estimate parameters by matching moments. These steps are not needed to define the firm’s dynamic programming problem, and they do not follow from solving the Bellman equation at one parameter vector. Hence the solver blocks are closely related, while the full estimation algorithms are not the same. $\square$

10.3 Is it suitable for the risky-debt problem?

The method is promising for the risky-debt problem, especially if the researcher wants to estimate parameters rather than merely compute one solution. The reason is that risky-debt models are expensive to solve repeatedly. A neural solution conditioned on parameters could amortize this cost: after training, one could evaluate policies and simulate moments for many parameter values quickly.

For the narrower goal of solving the risky-debt model in this note, the method is suitable only after addressing several gaps.

1. **Debt-pricing fixed point.** In risky-debt models, the risky debt price depends on future default, while default depends on equity value, and equity value depends on the debt price. Duarte and Fonseca (2026, Appendix A.3.4) handle this circularity with a lagged target network for bond prices during training, followed by grid refinement. This is a useful numerical device, but for our note it should be treated as an approximation strategy, not a proof that the debt-pricing fixed point has been solved. We would still need out-of-sample pricing residuals.
2. **Default boundary and nonsmooth payoffs.** Duarte and Fonseca (2026, Section 3.1.3 and Appendix A.4) smooth indicator functions during training and then refine using exact nonsmooth payoffs. This is natural for default and financing-cost kinks. The gap is that smoothing can bias the learned boundary, while grid refinement can become expensive as the number of state and control variables grows.

3. **Local versus global policy improvement.** A neural policy improvement step can improve the policy where gradients point in the right direction, but the risky-debt objective can be nonconcave. A policy network may therefore settle near a local optimum. This note’s direct policy-check residuals and the Duarte–Fonseca grid-refinement idea are both ways to test for better actions away from the network choice.
4. **Parameter conditioning increases dimension.** Conditioning the value and policy networks on structural parameters is powerful for estimation, but it enlarges the input space. For the risky-debt model, parameters such as default costs, tax rates, adjustment-cost parameters, and financing-cost parameters may interact strongly. The network must be accurate not just at one parameter vector, but across the whole training region.
5. **Moment networks add a second approximation.** If we use the full structural-estimation pipeline, then parameter estimates depend not only on value and policy approximation errors, but also on moment-network errors. The cross-validation and minimum-loss diagnostics in Duarte and Fonseca (2026, Sections 3.3 and 3.6) are important, but they do not remove the need to validate estimates by direct simulation from the solved model.
6. **General-equilibrium pseudo prices are not the same as risky debt price functions.** Treating a wage as a pseudo parameter works because the wage is a low-dimensional equilibrium object in their general equilibrium extension. In the risky-debt model, the debt price or risky rate is a function of state and promised actions, such as $q(z, k', b')$ or $\tilde{r}(z, k', b')$. That object cannot usually be replaced by one scalar pseudo parameter. The lender zero-profit condition still needs to be enforced as a functional equation.

Thus, for solving a fixed risky-debt model, Duarte–Fonseca should be viewed as a close cousin of the direct Bellman/policy approach already in this note. For estimating a risky-debt model, it suggests an important extension: condition the solution on parameters, simulate moments, learn a differentiable moment map, and then estimate parameters by matching moments. The main open work is to make the default boundary, debt-pricing fixed point, nonconvex policy choice, and moment surrogate errors visible in diagnostics rather than hidden inside a single training loss.

11 Parameter-conditioned Maliar–Maliar–Winant solvers

The preceding comparison was for one fixed parameter vector. But the neural residual method of Maliar et al. (2021) does not require the unknown functions to depend only on economic states. We can enlarge the input of the neural networks so that model parameters enter just like additional inputs. This gives a *parameter-conditioned* solver.

Let

$$\omega = (\delta, \tau, \alpha, \rho, \sigma_\varepsilon, \text{adjustment-cost parameters, financing-cost parameters}, \dots)$$

collect the structural parameters. Instead of learning one value function

$$V(k, b, z)$$

for one fixed ω , we learn a family of value functions with one neural network:

$$\hat{V}_\theta(k, b, z, \omega).$$

Likewise, we learn parameter-conditioned policy and pricing functions

$$\hat{h}_\theta^k(k, b, z, \omega), \quad \hat{h}_\theta^b(k, b, z, \omega), \quad \hat{r}_\theta(z, k', b', \omega). \quad (61)$$

For each fixed value of ω , these functions should satisfy the same Bellman, default, policy-optimality, and lender-pricing equations as before. The training loss therefore becomes an average not only over states, but also over parameters:

$$\mathcal{L}(\theta) = \mathbb{E}_{(k, b, z, \omega) \sim \mu} \left[\mathcal{B}_\theta(k, b, z, \omega)^2 + \mathcal{P}_\theta(k, b, z, \omega)^2 + \mathcal{M}_\theta(k, b, z, \omega)^2 \right], \quad (62)$$

where μ is the distribution from which we draw training points. Here $\mathcal{B}_\theta$ is the Bellman/default residual, $\mathcal{P}_\theta$ is the debt-pricing residual, and $\mathcal{M}_\theta$ is the policy-improvement or direct policy-check residual, now evaluated at the parameter vector ω .

Proposition 14 (Parameter conditioning gives the same solution block). *Suppose a parameter-conditioned neural solver and the value/policy block of Duarte and Fonseca (2026) use the same state variables, the same structural parameter vector, the same feasible action set, and the same lender zero-profit pricing equation. Then both methods are trying to approximate the same parameter-indexed fixed point*

$$\omega \mapsto (V_\omega, h_\omega^k, h_\omega^b, \tilde{r}_\omega).$$

Thus, at the model-solution level, a parameter-conditioned Maliar–Maliar–Winant residual solver is the same kind of object as the Duarte–Fonseca neural solution block. The Duarte–Fonseca paper then adds simulation, moment networks, estimation, and identification diagnostics on top of this block.

Proof. Fix any parameter vector ω . Once ω is fixed, the parameter-conditioned networks in eq. (61) become ordinary functions of the economic state and promised actions. Setting the residuals in eq. (62) to zero at this fixed ω gives exactly the risky-debt Bellman/default equation, the lender-pricing equation, and the policy-optimality condition for that parameter vector. Duarte and Fonseca’s value/policy block also seeks a value function and policies indexed by parameters (Duarte and Fonseca, 2026, Section 3.1 and eq. (13)). Therefore both solve the same family of parameter-specific dynamic programs. The remaining blocks of their algorithm act after the model is solved: they simulate panels, learn the parameter-to-moment map, and estimate parameters. Those blocks are not part of the Bellman fixed point itself. $\square$

This proposition is the clean answer to the question: yes, once the Maliar–Maliar–Winant style solver is extended to take model parameters as inputs, it becomes the same type of neural *solution* object as the first block of Duarte and Fonseca. The equality is not an equality of every numerical detail. A residual solver may train all residuals jointly; neural policy iteration may alternate policy evaluation and improvement. One may concatenate states and parameters; another may use an architecture in which parameters modulate the state network. These are different numerical designs for approximating the same parameter-indexed solution map.

11.1 Why parameter sampling is difficult

The difficult new object in eq. (62) is the sampling distribution μ . For a fixed model, we only have to decide which states (k, b, z) to sample. For a parameter-conditioned model, we must decide which pairs

$$(k, b, z, \omega)$$

to sample. This is much harder.

1. **The dimension grows quickly.** If the original risky-debt model has three state variables and we add eight parameters, the input space has eleven dimensions. A training set that looks dense in three dimensions can be extremely sparse in eleven dimensions. This is the usual curse of dimensionality, even though we are using neural networks rather than grids. Duarte and Fonseca explicitly emphasize this enlargement of the input space in their solution block (Duarte and Fonseca, 2026, Section 3.1).
2. **Parameters should not be sampled independently without thought.** A rectangular box such as

$$\omega_j \in [\underline{\omega}_j, \bar{\omega}_j]$$

is easy to sample from, but many combinations inside the box may be economically unimportant, weakly identified, or numerically pathological. For example, high debt benefits, low recovery, high productivity volatility, and low adjustment costs may create much more default activity than other combinations. Uniform sampling over a wide box can spend most computation on parameter regions that will never matter for estimation.

3. **The relevant state region depends on parameters.** The typical range of capital, debt, and productivity is not the same for all ω . If shocks are very persistent, the relevant range of z may be wider. If adjustment costs are high, capital moves less. If default costs are low, high-debt states may be more common. Therefore the state-sampling rule should usually depend on the sampled parameter vector:

$$\omega \sim \mu_\omega, \quad (k, b, z) \sim \mu_x(\cdot \mid \omega).$$

Duarte and Fonseca use parameter-dependent state bounds in their training procedure (Duarte and Fonseca, 2026, Appendix A.2 and Appendix A.3.1).

4. **Some parameters change the shape of the solution sharply.** Near a default boundary, a small change in recovery, volatility, or tax benefits can move the boundary a lot. A network trained on a coarse parameter sample may average across these changes and produce a smoothed default rule in the wrong place. This is especially dangerous for risky debt because the default boundary feeds back into the debt price.
5. **Accuracy is not needed equally everywhere.** If the goal is estimation, the solver must be accurate near parameter values that match the data. It may be less important to solve remote parameter regions precisely. But we do not know the important region before estimating the model. This is the chicken-and-egg problem in parameter sampling. Duarte and Fonseca address this by starting from wide parameter bounds, sampling parameters inside those bounds, and then narrowing the bounds using minimum-loss functions with safeguards against excluding plausible or weakly identified regions (Duarte and Fonseca, 2026, Sections 3.5–3.6 and Appendix A.9).
6. **Weak identification makes shrinking dangerous.** Suppose a parameter barely affects the targeted moments. Then many values of that parameter fit the data almost equally well. If we shrink the training region too aggressively, we may accidentally throw away valid parameter values. Parameter sampling is therefore not only a numerical issue; it is also an identification issue.

A practical parameter-conditioned solver should therefore report diagnostics by parameter region. For example, after training, one should evaluate Bellman residuals, pricing residuals, direct policy

checks, and simulation moments on a held-out sample of parameter vectors. It is not enough to report one average loss over all sampled (k, b, z, ω) . A small average can hide large errors near default boundaries, high-leverage regions, or weakly identified parameter directions.

12 An alternative estimation route: state-space models, filtering, and HMC

Duarte and Fonseca’s approach estimates parameters by learning a differentiable map from parameters to simulated moments. There is another natural route. After we solve the risky-debt model, the solved policy functions give laws of motion for capital and debt. Together with the shock process, these laws of motion can be treated as a *state-space model*. Then we can estimate structural parameters using filtering and Bayesian simulation methods such as Hamiltonian Monte Carlo (HMC).

This is the same broad logic used in many likelihood-based dynamic models: first solve the economic decision problem, then use the solved model as the transition law for hidden economic states. State-space likelihood methods are a standard way to handle latent states (Durbin and Koopman, 2012). HMC is a standard gradient-based method for sampling continuous posterior distributions (Neal, 2011). When the state-space model is nonlinear or non-Gaussian, particle filters and particle MCMC methods are often used (Andrieu et al., 2010).

12.1 From solved policies to a state-space model

Let ω denote the structural parameter vector. Suppose that, for each ω , the neural solver gives approximate policy and pricing functions

$$\hat{h}_\omega^k(k, b, z), \quad \hat{h}_\omega^b(k, b, z), \quad \hat{r}_\omega(z, k', b').$$

For firm i at date t , define the latent state

$$x_{i,t} = (k_{i,t}, b_{i,t}, z_{i,t}, a_{i,t}),$$

where $a_{i,t} \in \{0, 1\}$ is an indicator for whether the firm is active. If the firm is active, the solved policies imply

$$k_{i,t+1} = \hat{h}_\omega^k(k_{i,t}, b_{i,t}, z_{i,t}), \tag{63}$$

$$b_{i,t+1} = \hat{h}_\omega^b(k_{i,t}, b_{i,t}, z_{i,t}), \tag{64}$$

$$\log z_{i,t+1} = \rho \log z_{i,t} + \sigma_\varepsilon \varepsilon_{i,t+1}, \quad \varepsilon_{i,t+1} \sim N(0, 1). \tag{65}$$

Default is determined by the solved continuation value. For example, using the going-concern value $\hat{G}_\omega$ from definition 2, one can write the next-period default indicator as

$$d_{i,t+1} = \mathbf{1}\{\hat{G}_\omega(k_{i,t+1}, b_{i,t+1}, z_{i,t+1}) < 0\}. \tag{66}$$

If the firm defaults, the econometrician must specify what happens in the data: the firm may exit the sample, be replaced by a new draw, or remain observed with a default flag. This choice is not part of the Bellman equation; it is part of the statistical observation model.

Let $y_{i,t}$ denote observed data, such as book assets, leverage, investment, equity issuance, market value, or a default indicator. A simple measurement equation is

$$y_{i,t} = m_\omega(x_{i,t}) + u_{i,t}, \quad u_{i,t} \sim N(0, \Sigma_u), \tag{67}$$

where m_ω maps the model state into observable quantities. If the data include a binary default observation, then that component is better written as a Bernoulli measurement equation rather than a Gaussian one.

Proposition 15 (Solved risky-debt model as a state-space model). *Fix a parameter vector ω . If the policy functions, debt-pricing function, shock law, default rule, and measurement equation are specified, then the risky-debt model defines a state-space model*

$$p_\omega(x_{t+1} | x_t), \quad p_\omega(y_t | x_t).$$

Therefore the likelihood of a time-series or panel dataset can be written as

$$p_\omega(y_{1:T}) = \int p_\omega(x_1) \prod_{t=1}^T p_\omega(y_t | x_t) \prod_{t=1}^{T-1} p_\omega(x_{t+1} | x_t) dx_{1:T}. \quad (68)$$

Proof. Equations eqs. (63) to (65) describe how the hidden state evolves. The next state depends on the current state and the new shock, so it defines a transition law $p_\omega(x_{t+1} | x_t)$, allowing for exit, replacement, or default flags if those are included in the state. Some parts of this law may be deterministic: for example, conditional on the current state and the shock, next period's capital and debt are pinned down by the policy functions. Equation eq. (67) describes the distribution of observables conditional on the hidden state, so it defines $p_\omega(y_t | x_t)$. The joint density of states and observations is therefore the initial density times all transition densities times all measurement densities. Integrating out the unobserved states gives eq. (68). $\square$

12.2 Filtering and HMC

The filtering recursion updates beliefs about the hidden state as data arrive:

$$p_\omega(x_t | y_{1:t-1}) = \int p_\omega(x_t | x_{t-1}) p_\omega(x_{t-1} | y_{1:t-1}) dx_{t-1}, \quad (69)$$

$$p_\omega(x_t | y_{1:t}) \propto p_\omega(y_t | x_t) p_\omega(x_t | y_{1:t-1}). \quad (70)$$

In a linear Gaussian model these equations reduce to the Kalman filter. The risky-debt model is not linear Gaussian because policies are nonlinear and default creates a discrete event. Therefore a particle filter is usually the more natural filtering tool.

For Bayesian estimation, place a prior $p(\omega)$ on the structural parameters. The posterior is

$$p(\omega | y_{1:T}) \propto p(\omega) p_\omega(y_{1:T}). \quad (71)$$

If $p_\omega(y_{1:T})$ and its gradient can be evaluated accurately and smoothly, HMC can sample from eq. (71). This is attractive because HMC uses gradients to move efficiently through a high-dimensional continuous parameter space.

12.3 Can we do this for the risky-debt model?

Yes, but it is harder than writing down eq. (71). The main difficulties are the following.

1. **The likelihood needs more than the solved Bellman equation.** Solving the firm problem gives policy functions. It does not by itself give measurement errors, replacement rules after default, sample-selection rules, or the distribution of initial firm states. These are statistical modeling choices.

2. **Default makes the model non-Gaussian and partly discrete.** HMC works naturally for continuous differentiable posterior densities. A hard default indicator creates kinks and discrete events. One can either smooth the default rule for differentiable HMC, integrate default events with a filter, or use particle MCMC methods that are designed for nonlinear and non-Gaussian state-space models.
3. **A particle-filter likelihood is noisy.** Particle filters give a simulation-based likelihood estimate. That estimate can be useful, but its Monte Carlo noise can make gradient-based HMC unstable unless the filter is carefully designed. Particle MCMC methods are a more direct fit when the likelihood is estimated by particles (Andrieu et al., 2010).
4. **The neural solution depends on parameters.** In a likelihood or posterior evaluation at a new ω , the policies $\hat{h}_\omega^k$, $\hat{h}_\omega^b$, and prices $\hat{r}_\omega$ must be accurate at that ω . This again points toward a parameter-conditioned solver and held-out residual diagnostics by parameter region.
5. **Approximation error enters the likelihood.** The likelihood in eq. (68) is the likelihood of the *approximated* solved model. If the neural policy is wrong near the default boundary, the filter may fit the data by moving parameters in a biased direction. A serious estimation exercise should therefore report both filtering diagnostics and solver residual diagnostics.

Thus the answer is yes in principle: after solving the risky-debt model, we can turn it into a state-space model and estimate it with filtering plus HMC or particle MCMC. This is different from Duarte and Fonseca’s moment-network route. Their route learns a parameter-to-moment map and matches moments. The state-space route writes a likelihood for the time-series or panel data and estimates from that likelihood. The price of the likelihood route is that we must specify a full observation model and handle the nonlinear, nonsmooth, and partly discrete default structure.

13 A full-scale state-space specification

This section gives one complete specification. It is not the only possible specification, but it is complete enough that the likelihood is well defined. It is designed to address the five problems listed above.

13.1 Data and parameters

Suppose we observe a panel of firms $i = 1, \dots, N$ over dates $t = 1, \dots, T_i$. Let the observed vector be

$$y_{i,t} = (\log A_{i,t}^{obs}, L_{i,t}^{obs}, I_{i,t}^{obs}/A_{i,t}^{obs}, Q_{i,t}^{obs}, D_{i,t}^{obs}),$$

where A^{obs} is an accounting measure of assets, L^{obs} is leverage, I^{obs}/A^{obs} is the investment rate, Q^{obs} is a market-value or valuation ratio, and $D^{obs} \in \{0, 1\}$ is an observed default or exit indicator. If some components are missing, the likelihood simply omits those components for that firm-date.

Let the estimated parameter vector be

$$\phi = (\omega, \Sigma_y, \mu_0, \Sigma_0, \pi_{01}, \pi_{10}, \kappa).$$

Here ω contains the economic parameters in the risky-debt model, Σ_y contains measurement-error variances, (μ_0, Σ_0) describe the initial distribution of latent states, π_{01} and π_{10} are default-observation error probabilities, and $\kappa > 0$ is a smoothing parameter used only in the differentiable approximation described below. If one uses the exact discrete default model, κ can be omitted.

13.2 Latent state and initial distribution

For each firm, define the latent state

$$x_{i,t} = (k_{i,t}, b_{i,t}, z_{i,t}, a_{i,t}),$$

where $a_{i,t} = 1$ means the firm is active and $a_{i,t} = 0$ means the firm has defaulted or exited. The initial state distribution is

$$\log k_{i,1} \sim N(\mu_{k,0}, \sigma_{k,0}^2), \quad (72)$$

$$b_{i,1}/k_{i,1} \sim N(\mu_{\ell,0}, \sigma_{\ell,0}^2), \quad (73)$$

$$\log z_{i,1} \sim N(\mu_{z,0}, \sigma_{z,0}^2), \quad (74)$$

$$a_{i,1} = 1. \quad (75)$$

These normal distributions can be replaced by truncated normal distributions if the researcher wants to rule out economically impossible initial states.

13.3 Solved policy transition

For a proposed economic parameter vector ω , use the parameter-conditioned neural solver to obtain

$$\hat{V}_\omega, \quad \hat{G}_\omega, \quad \hat{h}_\omega^k, \quad \hat{h}_\omega^b, \quad \hat{r}_\omega.$$

For an active firm, draw a productivity innovation $\varepsilon_{i,t+1} \sim N(0, 1)$ and set

$$k_{i,t+1}^* = \hat{h}_\omega^k(k_{i,t}, b_{i,t}, z_{i,t}), \quad (76)$$

$$b_{i,t+1}^* = \hat{h}_\omega^b(k_{i,t}, b_{i,t}, z_{i,t}), \quad (77)$$

$$\log z_{i,t+1}^* = \rho \log z_{i,t} + \sigma_\varepsilon \varepsilon_{i,t+1}. \quad (78)$$

The exact model default indicator is

$$d_{i,t+1}^* = \mathbf{1}\{\hat{G}_\omega(k_{i,t+1}^*, b_{i,t+1}^*, z_{i,t+1}^*) < 0\}. \quad (79)$$

If $d_{i,t+1}^* = 0$, then

$$x_{i,t+1} = (k_{i,t+1}^*, b_{i,t+1}^*, z_{i,t+1}^*, 1).$$

If $d_{i,t+1}^* = 1$, use an absorbing exit rule:

$$a_{i,t+1} = 0.$$

All later observations for that firm are treated as missing except the default or exit indicator. This absorbing-exit rule is natural for firm-level data in which defaulted firms leave the sample. If instead the model is being simulated as a stationary artificial panel, replace the defaulted firm by a new draw from the initial distribution. The important point is that the rule must be stated before estimation.

13.4 Measurement equations

For an active firm, define model-implied observables by

$$\log A_{i,t}^{mod} = \log k_{i,t}, \quad (80)$$

$$L_{i,t}^{mod} = b_{i,t}/k_{i,t}, \quad (81)$$

$$(I/A)_{i,t}^{mod} = \frac{\hat{h}_{\omega}^k(k_{i,t}, b_{i,t}, z_{i,t}) - (1 - \delta)k_{i,t}}{k_{i,t}}, \quad (82)$$

$$Q_{i,t}^{mod} = \frac{\hat{V}_{\omega}(k_{i,t}, b_{i,t}, z_{i,t}) + b_{i,t}}{k_{i,t}}. \quad (83)$$

The last equation uses equity plus debt divided by capital as a simple model-implied valuation ratio. If the data define Q differently, then this measurement equation should be changed to match the data definition.

The continuous observations satisfy

$$y_{i,t}^{cont} = m_{\omega}^{cont}(x_{i,t}) + u_{i,t}, \quad u_{i,t} \sim N(0, \Sigma_y). \quad (84)$$

For the binary default observation, allow small classification errors:

$$\begin{aligned} \Pr(D_{i,t+1}^{obs} = 1 \mid d_{i,t+1}^* = 0) &= \pi_{01}, \\ \Pr(D_{i,t+1}^{obs} = 0 \mid d_{i,t+1}^* = 1) &= \pi_{10}. \end{aligned} \quad (85)$$

These probabilities keep the likelihood from becoming exactly zero when data and model default dates disagree by a small amount.

13.5 Exact filtering specification

The exact-discrete version uses eq. (79) as written. A particle filter approximates

$$p_{\phi}(x_{i,t} \mid y_{i,1:t})$$

by particles $x_{i,t}^{(m)}$, $m = 1, \dots, M$. At each date:

1. Propagate each particle using the solved policy transition and a new shock draw.
2. Compute the observation likelihood from eqs. (84) and (85).
3. Reweight particles by the observation likelihood.
4. Resample if the effective sample size is too small.

This gives an estimated likelihood

$$\hat{p}_{\phi}(y_{i,1:T_i})$$

for each firm. If firms are conditionally independent given ϕ , the panel likelihood is

$$\hat{p}_{\phi}(Y) = \prod_{i=1}^N \hat{p}_{\phi}(y_{i,1:T_i}). \quad (86)$$

The Bayesian posterior is

$$p(\phi \mid Y) \propto p(\phi) p_{\phi}(Y). \quad (87)$$

Because the likelihood is estimated by particles, the most faithful sampler is a particle MCMC method, such as particle marginal Metropolis-Hastings (Andrieu et al., 2010). This directly addresses the discreteness of default and the non-Gaussian state-space likelihood. Plain HMC is not the safest choice for this exact-discrete likelihood because resampling and hard default events make the likelihood rough.

13.6 Smoothed HMC specification

If the researcher wants HMC, use a differentiable approximation. Replace the hard default indicator by the smooth default probability

$$p_{\kappa,i,t+1}^D = \frac{1}{1 + \exp\{\widehat{G}_\omega(k_{i,t+1}^*, b_{i,t+1}^*, z_{i,t+1}^*)/\kappa\}}. \quad (88)$$

When $\widehat{G}_\omega$ is very negative, the default probability is close to one. When $\widehat{G}_\omega$ is very positive, it is close to zero. As $\kappa \downarrow 0$, this approaches the hard default rule.

In the smoothed likelihood, use

$$D_{i,t+1}^{obs} \sim \text{Bernoulli}(p_{\kappa,i,t+1}^D)$$

or combine $p_{\kappa,i,t+1}^D$ with the classification-error probabilities π_{01} and π_{10} . If the resulting filtering likelihood is differentiable in ϕ , HMC can target the approximate posterior

$$\tilde{p}_\kappa(\phi \mid Y) \propto p(\phi) \tilde{p}_{\phi,\kappa}(Y). \quad (89)$$

The smoothing parameter is not harmless. A serious analysis should repeat the estimation for smaller κ and check whether posterior conclusions are stable. If conclusions change materially as κ changes, the smoothed HMC posterior is a numerical approximation, not reliable evidence about the exact default model.

13.7 How this specification addresses the five problems

1. **More than the Bellman equation.** The model now specifies initial states, transitions, default/exit, measurement errors, default observation errors, and missing-data handling. The likelihood is therefore a statistical object, not merely a solved dynamic program.
2. **Default discreteness.** The exact version uses a particle filter and particle MCMC, which are designed for nonlinear and non-Gaussian state-space models. The HMC version uses a smooth default probability and must be checked against the exact version.
3. **Particle-filter noise.** The exact version uses the particle-filter likelihood inside particle MCMC rather than pretending that the likelihood is deterministic. In practice one should increase the number of particles until the log-likelihood estimate is stable enough for the sampler being used.
4. **Parameter-dependent neural solution.** The solver must be parameter-conditioned on ω . During estimation, the posterior draws of ω should remain inside the region where held-out Bellman, pricing, and policy-check residuals are small.

5. **Approximation error.** Before accepting posterior draws, compute a solver diagnostic such as

$$R_{0.95}(\omega) = 95\text{th percentile of held-out solver residuals at } \omega.$$

If $R_{0.95}(\omega)$ is too large in posterior regions, the solution network must be retrained or refined there. The likelihood should not be trusted in parameter regions where the economic model has not actually been solved accurately.

Proposition 16 (A well-defined full-scale estimator). *Once the structural parameters, solved policy functions, initial-state law, transition law, default/exit rule, measurement equations, and prior are specified as above, the risky-debt model defines either an exact-discrete particle-filter likelihood $p_\phi(Y)$ or a smoothed likelihood $\tilde{p}_{\phi,\kappa}(Y)$. Therefore estimation by particle MCMC or by HMC on the smoothed posterior is mathematically well defined.*

Proof. The initial-state law gives $p_\phi(x_{i,1})$. The solved policy transition, shock law, and default/exit rule give $p_\phi(x_{i,t+1} \mid x_{i,t})$. The continuous and binary measurement equations give $p_\phi(y_{i,t} \mid x_{i,t})$. Multiplying these terms and integrating over latent states gives a likelihood. Combining the likelihood with a prior gives a posterior. Particle MCMC targets the posterior associated with the exact-discrete state-space likelihood, while HMC targets the posterior associated with the differentiable smoothed likelihood. $\square$

14 Likelihood filtering versus moment matching

We now have two estimation routes for the risky-debt model.

The *moment-matching route* solves the model for many parameter values, simulates firm panels, computes selected moments, learns a differentiable moment map, and estimates parameters by matching data moments. This is the route emphasized by Duarte and Fonseca (2026, Section 3). In our notation, it estimates

$$\hat{\omega}_{\text{MM}} \in \arg \min_{\omega} (\hat{m} - g(\omega))^\top W (\hat{m} - g(\omega)), \quad (90)$$

where $\hat{m}$ are empirical moments and $g(\omega)$ is the model-implied moment function or its neural approximation.

The *likelihood-filtering route* solves the model, embeds the solved policies in a state-space model, filters latent states, and estimates from the likelihood. In Bayesian form it estimates the posterior

$$p(\phi \mid Y) \propto p(\phi) p_\phi(Y), \quad (91)$$

where $p_\phi(Y)$ is obtained by a Kalman filter in special linear Gaussian cases, by a particle filter in nonlinear non-Gaussian cases, or by a smoothed differentiable approximation when using HMC.

Proposition 17 (Different statistical objects). *Moment matching and likelihood filtering use the same solved economic model, but they estimate different statistical objects. Moment matching compares a small vector of selected features of the data with the same features in simulated data. Likelihood filtering specifies a full probability law for the observed panel and uses that law to evaluate the probability of the entire observed path.*

Proof. In eq. (90), the data enter only through $\hat{m}$. If two datasets have the same selected moments, the objective gives them the same value. In eq. (91), the data enter through the full likelihood $p_\phi(Y)$. The likelihood depends on the order of observations, the timing of default, the path of latent states, and measurement errors. Thus the two routes use the solved model differently: one through selected moments, the other through a probability model for the full observed history. $\square$

14.1 Advantages of moment matching

Moment matching is often the more forgiving route.

1. **It does not require a complete observation model.** The researcher chooses moments such as mean leverage, investment volatility, default frequency, or autocorrelations. One does not need to specify every measurement-error distribution or the exact probability of every observed firm path.
2. **It can target economically important features directly.** If leverage, default frequency, and investment moments are the main objects of interest, the estimator can put them directly in $\hat{m}$. This is useful for teaching and for empirical work where the model is intended to explain selected corporate-finance facts.
3. **It can be computationally efficient after the surrogate is trained.** Once the moment network $g(\omega)$ is accurate, evaluating eq. (90) is fast. This is the central computational advantage of the Duarte–Fonseca approach.
4. **It is less exposed to small measurement-model mistakes.** A full likelihood can be very sensitive to a badly specified measurement error. Moment matching can be more robust if the selected moments are well chosen.
5. **Identification diagnostics can be visual.** Minimum-loss functions, as in Duarte and Fonseca (2026, Sections 3.5–3.6), let the researcher ask whether each parameter is pinned down by the chosen moments.

The main weaknesses are also clear. Moment matching throws away information not contained in the selected moments. It can identify only what the chosen moments are able to identify. It also introduces a second approximation if $g(\omega)$ is learned by a neural moment network. Finally, the choice of weighting matrix W and the choice of moments can materially affect the answer.

14.2 Advantages of likelihood filtering and HMC

The likelihood route is more ambitious.

1. **It uses the timing of the data.** A state-space likelihood uses the sequence of leverage, investment, valuation, and default observations, not just their averages or autocorrelations. Timing is especially valuable for default because the model predicts when a firm crosses the default boundary.
2. **It handles latent states explicitly.** Filtering gives a distribution over unobserved capital, productivity, and default risk at each date. This can be useful for posterior predictive checks and for studying hidden firm quality.
3. **It gives a full uncertainty statement.** In the Bayesian version, HMC or particle MCMC produces posterior draws of parameters and latent states. This gives uncertainty intervals for structural parameters, policies, default probabilities, and counterfactuals.
4. **It can use irregular and partially missing observations.** A state-space likelihood can omit missing components date by date while still updating beliefs with the observed components.

5. **It tests the model more harshly.** Because the likelihood tries to explain the whole observed path, it exposes misspecification that moment matching may hide.

The costs are substantial. The researcher must specify a full measurement model, initial distribution, exit rule, and default-observation process. The filter can be expensive for large panels. Hard default makes the likelihood nonsmooth and non-Gaussian. Plain HMC is suitable only for a differentiable posterior, so the exact default model usually points toward particle MCMC, while HMC points toward a smoothed approximation whose sensitivity must be checked. Finally, if the neural solution has errors near the default boundary, the likelihood can be biased in a very precise-looking way.

14.3 Side-by-side comparison

The practical comparison is as follows.

- **Object being matched.** Moment matching matches selected statistics. Filtering matches the probability law of the observed paths.
- **Data requirement.** Moment matching can work with repeated cross sections or a panel summarized by moments. Filtering is most valuable when firm-level histories and default timing are observed.
- **Modeling burden.** Moment matching needs moment definitions and a weighting matrix. Filtering needs a full observation model, latent-state transition, initial distribution, and default/exit rule.
- **Computation.** Moment matching is expensive while building the simulation and moment-network dataset, but cheap afterward. Filtering is expensive at every likelihood evaluation unless the filter is very efficient.
- **Default treatment.** Moment matching can target default frequency and related moments without explaining every default date. Filtering must assign probabilities to default timing, which is more informative but also more fragile.
- **Uncertainty.** Moment matching can use asymptotic standard errors, bootstrap, or sensitivity across moment-network folds. Bayesian filtering gives posterior uncertainty directly, conditional on the full probabilistic specification.
- **Main failure mode.** Moment matching can fit chosen moments while missing important dynamics. Filtering can look precise while being wrong because the observation model or neural solution is misspecified.

14.4 Which one should we use?

For a first empirical risky-debt project, moment matching is usually the safer starting point. It is easier to explain, easier to debug, and less dependent on strong assumptions about measurement errors and exact firm histories. It is especially attractive when the goal is to match corporate-finance facts such as average leverage, investment volatility, issuance frequency, and default rates.

The likelihood-filtering route is attractive when the data contain reliable firm-level histories and the researcher cares about latent states or default timing. It is also useful when one wants posterior distributions for counterfactuals, not just a point estimate. For the risky-debt model, the most credible likelihood implementation is likely a two-step comparison:

1. Estimate first by moment matching to find a plausible parameter region.
2. Train or refine the parameter-conditioned neural solution in that region.
3. Run the exact-discrete particle-filter estimator as the benchmark likelihood method.
4. Run smoothed HMC as a faster differentiable approximation.
5. Check whether the smoothed HMC posterior agrees with the exact particle-filter benchmark and whether solver residuals are small throughout the posterior region.

This hybrid workflow uses the strengths of both approaches. Moment matching finds the region of the parameter space that deserves attention. Filtering then asks the sharper question: can the solved model explain the actual paths of firms, including the timing of default?

15 The all-in-one expectation idea

Maliar et al.’s all-in-one integration idea is easiest to understand through a small algebra fact. Suppose $f(\varepsilon)$ is a random number depending on a shock ε . Let ε_1 and ε_2 be independent draws from the same distribution.

Proposition 18 (Two-draw identity). *If ε_1 and ε_2 are independent and identically distributed, then*

$$(\mathbb{E}[f(\varepsilon)])^2 = \mathbb{E}[f(\varepsilon_1)f(\varepsilon_2)]. \quad (92)$$

Proof. Because the two shocks are independent, the expectation of the product equals the product of expectations:

$$\mathbb{E}[f(\varepsilon_1)f(\varepsilon_2)] = \mathbb{E}[f(\varepsilon_1)]\mathbb{E}[f(\varepsilon_2)].$$

Because the two shocks have the same distribution,

$$\mathbb{E}[f(\varepsilon_1)] = \mathbb{E}[f(\varepsilon_2)] = \mathbb{E}[f(\varepsilon)].$$

Substituting gives eq. (92). □

This identity explains why two independent shock draws can estimate certain squared expected residuals. Maliar et al. use this idea to reduce the cost of numerical integration in Bellman and Euler residual objectives (Maliar et al., 2021, Definitions 2.9 and 2.10). In the risky-debt model, this identity is directly useful only for squared residual components that are linear in a conditional expectation before squaring. The full risky-debt objective also contains nonlinear pieces, such as the Fischer-Burmeister expression in eq. (27), and possibly a smoothed default classifier. For those pieces, a practical implementation should use mini-batch inner averages or a separately justified reformulation.

16 A concrete algorithm

A practical version of the method is as follows.

1. Choose neural-network architectures for $\hat{V}_\theta$, $\hat{h}_\theta^k$, $\hat{h}_\theta^b$, and $\hat{r}_\theta$.

2. Decide which constraints will be hard-coded in the output layer. For example, use sigmoid transformations for bounded choices k' and b' , and use softplus transformations for positive quantities such as gross risky returns. Keep Bellman/default residuals for the default boundary, because default is a regime choice rather than an ordinary output bound.
3. Draw an initial batch of states $s_i = (k_i, b_i, z_i)$. These can be drawn from a broad rectangle or simulated from a preliminary policy.
4. For each state, compute the policy $(\hat{k}'_i, \hat{b}'_i)$.
5. Draw future shocks using
$$\log z'_i = \rho \log z_i + \varepsilon'_i.$$
6. Compute the continuation gate C_θ , the interior gate I_θ , and the product $S_\theta = C_\theta I_\theta$.
7. On states with $S_\theta \approx 1$, compute the Euler residuals $\hat{\mathcal{E}}_\theta^k$ and $\hat{\mathcal{E}}_\theta^b$. On states near default or action bounds, compute the Bellman/default residual $\mathcal{B}_\theta$ and any KKT residuals $\mathcal{K}_\theta$. If the relevant action bounds are hard-coded by output transformations, the corresponding KKT residuals can be omitted.
8. For every positive debt choice $\hat{b}'_i > 0$, compute the pricing residual $\mathcal{P}_\theta$, because the risky rate must make lenders break even whether the firm is close to default or safely solvent.
9. Either minimize the empirical hybrid loss $\mathcal{L}_N^{\text{hybrid}}(\theta)$ by stochastic gradient descent or Adam, or use the hard-constrained hybrid loss $\mathcal{L}_N^{\text{hard-hybrid}}(\theta)$ when feasibility is enforced by construction. Alternatively, split the same residuals into separate objectives as in section 9.1, compute one gradient per objective, and update the network with a multi-objective aggregate such as MGDA. A direct policy-maximization loss $\mathcal{L}_N(\theta)$ can also be used, but it usually requires sampling many alternative actions.
10. Resimulate states using the updated policy and repeat.
11. Assess accuracy on fresh states not used in training. Report Bellman residuals, Euler residuals by region, KKT residuals near bounds, pricing residuals, MOO diagnostics when MOO is used, default classification behavior, direct checks against random alternative actions, and simulated financial moments.

17 What this derivation does and does not prove

The derivation proves a mathematical reformulation: the risky-debt Bellman problem, the default option, and the zero-profit debt-pricing equation can be written as residual equations suitable for the deep-learning framework of [Maliar et al. \(2021\)](#). It also shows why the risky-debt case needs more residuals than the basic Section 3.1 investment model: the risky rate $\tilde{r}$ must be learned or solved jointly with the value and policy functions.

The derivation does not prove that a neural network trained with a particular optimizer will converge to the true economic solution. Neural networks are approximating families, and stochastic gradient methods are numerical optimization tools. Multi-objective training can reduce sensitivity to fixed loss weights, but it is still a numerical optimizer with design choices: objective grouping, normalization, learning rate, aggregation rule, and stopping criteria all matter. Euler and KKT residuals are local necessary conditions, so they should be supplemented by direct checks against

alternative actions when global policy optimality matters. The right scientific claim after training is therefore not “the model is solved exactly.” The right claim is: the trained functions solve the model approximately on the tested state region, with approximation quality measured by out-of-sample Bellman, pricing, Euler-by-region, KKT, direct policy-check, multi-objective, and simulation diagnostics.

References

- Désidéri, Jean-Antoine. 2012. Multiple-gradient descent algorithm (MGDA) for multiobjective optimization. *Comptes Rendus Mathématique* 350(5–6):313–318. DOI: [10.1016/j.crma.2012.03.014](https://doi.org/10.1016/j.crma.2012.03.014).
- Andrieu, Christophe, Arnaud Doucet, and Roman Holenstein. 2010. Particle Markov chain Monte Carlo methods. *Journal of the Royal Statistical Society: Series B* 72(3):269–342. DOI: [10.1111/j.1467-9868.2009.00736.x](https://doi.org/10.1111/j.1467-9868.2009.00736.x).
- Durbin, James, and Siem Jan Koopman. 2012. *Time Series Analysis by State Space Methods*. Second edition. Oxford University Press.
- Duarte, Victor, and Julia Fonseca. 2026. AI for structural estimation. NBER Working Paper No. 35283, May 2026. Available at: <https://www.nber.org/papers/w35283>.
- Hall-Hoffarth, Emmet. 2023. Non-linear approximations of DSGE models with neural-networks and hard-constraints. arXiv:2310.13436. Available at: <https://arxiv.org/abs/2310.13436>.
- Maliar, Lilia, Serguei Maliar, and Pablo Winant. 2021. Deep learning for solving dynamic economic models. *Journal of Monetary Economics*. DOI: [10.1016/j.jmoneco.2021.07.004](https://doi.org/10.1016/j.jmoneco.2021.07.004).
- Neal, Radford M. 2011. MCMC using Hamiltonian dynamics. In *Handbook of Markov Chain Monte Carlo*, edited by Steve Brooks, Andrew Gelman, Galin Jones, and Xiao-Li Meng. Chapman and Hall/CRC. Available at: <https://www.mcmchandbook.net/HandbookChapter5.pdf>.
- Sener, Ozan, and Vladlen Koltun. 2018. Multi-task learning as multi-objective optimization. In *Advances in Neural Information Processing Systems* (NeurIPS), pages 525–536. Available at: [NeurIPS proceedings](https://neurips.cc/paper_files/paper/2018/file/Sener-18.pdf).
- Strebulaev, Ilya A., and Toni M. Whited. 2012. Dynamic models and structural estimation in corporate finance. Working paper, revised December 2012. This note uses the December 2012 working-paper version for section and equation numbering. Available at SSRN: <https://ssrn.com/abstract=2091854>.